Name : Sahil Jakhariya
Roll No : 63
Class : D15C
Batch : C

MLDL EXPERIMENT 10
Optimizing Convolutional Autoencoders for Image Denoising

Aim:

To explore and implement Autoencoders for image denoising.

Dataset Source: https://www.kaggle.com/datasets/hojjatk/mnist-dataset

Dataset Description:

The MNIST dataset is the standard benchmark in computer vision for evaluating image processing models. It consists of grayscale images of handwritten digits (0–9), traditionally used for classification, but repurposed here for image reconstruction tasks.

In this experiment, the raw pixel data is extracted from the t10k-images file. Each image is a 28 × 28 grid, providing 784 features per sample. For the denoising task, these images are normalized and then artificially corrupted with Gaussian noise to create input-target pairs.

Feature Description:

Feature	Description	Type	Unit
Pixel Intensity	Grayscale value of each pixel	Numerical (Float)	0.0 – 1.0
Image Dimensions	Spatial resolution	Categorical	28 × 28
Noise Factor	Variance of Gaussian noise	Numerical	$\sigma = 0.5$
Reconstruction	Output pixel value	Numerical	0.0 – 1.0

The dataset is highly structured and clean, which allows the Autoencoder to focus on learning the latent manifold (underlying digit structure) rather than being distracted by complex background textures.

Theory:

Autoencoders are a class of unsupervised neural networks designed to learn efficient data representations by training the model to reconstruct its input. An autoencoder consists of two main components:

1. **Encoder** – Compresses the image into a latent representation
2. **Decoder** – Reconstructs the image from that compressed representation

For image denoising, the model is trained using noisy images as input and clean images as output. This is known as a **Denoising Autoencoder (DAE)**.

The use of noise during training forces the network to learn meaningful and robust features rather than memorizing data. Convolutional Autoencoders (CAE) are preferred because they capture spatial patterns like edges and textures effectively.

The model is trained by minimizing a **reconstruction loss** such as:

- Mean Squared Error (MSE)
- Binary Cross-Entropy (BCE)

Performance is evaluated using:

- **PSNR (Peak Signal-to-Noise Ratio)**
- **SSIM (Structural Similarity Index)**

Mathematical Formulation of the Algorithms:

1. Encoder Function

h = f(We * x' + be)

Where:

- x' = Noisy input image
- We = Encoder weights
- be = Bias
- f = Activation function (ReLU)
- h = Latent representation

2. Decoder Function

x_hat = g(Wd * h + bd)

Where:

- h = Encoded output
- Wd = Decoder weights
- bd = Bias
- g = Activation function (Sigmoid)
- x_hat = Reconstructed image

3. Reconstruction Loss (MSE)

L = (1/n) * SUM (xi - x_hat_i)^2

Where:

- xi = Original pixel
- x_hat_i = Reconstructed pixel
- n = Number of pixels

4. Performance Metrics

PSNR:
PSNR = 10 * log10(MAX_I^2 / MSE)

Where:

- MAX_I = Maximum pixel value (1 or 255)
- MSE = Mean Squared Error

SSIM:
SSIM = ((2 * mu_x * mu_y + C1) * (2 * sigma_xy + C2))
/ ((mu_x^2 + mu_y^2 + C1) * (sigma_x^2 + sigma_y^2 + C2))

Where:

- mu_x, mu_y = Mean values
- sigma_x^2, sigma_y^2 = Variance
- sigma_xy = Covariance
- C1, C2 = Constants

Algorithm Limitations:

- **Noise Over-fitting:** If the noise distribution in the test set differs significantly from the training set (e.g., Salt-and-Pepper vs. Gaussian), the model's performance drops sharply.
- **Detail Blurring:** Autoencoders tend to average out pixel values to minimize MSE, which often results in the loss of fine edges or high frequency details.
- **Data Dependency:** The algorithm requires "clean" ground truth pairs for training, which are often unavailable or expensive to obtain in real-world medical or satellite imaging.
- **Fixed Resolution:** Standard CNN-based Autoencoders are sensitive to input size and cannot easily process images of varying resolutions without

Methodology / Workflow:

I. **Data Extraction:**
Parse the .idx3-ubyte binary file into a 4D NumPy array of shape (10000, 28, 28, 1).

II. **Preprocessing:**
Normalize pixel values to the range [0, 1] and split the dataset into training and testing sets.

III. **Noise Injection:**
Apply Gaussian noise with a factor of 0.5 and clip the values to ensure they remain within the valid intensity range.

IV. **Model Building:**
Construct a Convolutional Autoencoder consisting of an Encoder (Conv2D, MaxPooling) and a Decoder (Conv2D, UpSampling).

V. **Hyperparameter Tuning:**
Use KerasTuner to optimize the number of filters and the learning rate across multiple trials.

VI. **Training:**
Train the optimized model using the Adam optimizer for 30 epochs to achieve proper convergence.

VII. **Evaluation:**
Compute performance metrics such as MSE, PSNR, and SSIM on the test dataset and visualize the results using "Noisy vs Denoised" image comparisons.

CODE:

```
import numpy as np

import struct # No longer strictly needed for data loading but kept if other parts might use it.

from sklearn.model_selection import \

train_test_split # No longer needed for main split, but might be used if custom splits are desired.

from tensorflow.keras import layers, models

import matplotlib.pyplot as plt

from skimage.metrics import \

peak_signal_noise_ratio, structural_similarity

import tensorflow as tf # Added for direct MNIST loading

# --- 0. DATA LOADING (using Keras datasets) ---

# The previous manual download had an HTTP 404 error.

# Using tf.keras.datasets.mnist.load_data() for robust data loading.

# Load the MNIST dataset directly from Keras

(x_train_full, _), (x_test_full, _) = tf.keras.datasets.mnist.load_data()

# Normalize and reshape the images

# For an autoencoder, we typically don't need the labels, so we use `_`.

x_train = x_train_full.astype('float32') / 255.

x_test = x_test_full.astype('float32') / 255.

# Reshape data to include channel dimension (28, 28, 1)

x_train = np.expand_dims(x_train, -1)

x_test = np.expand_dims(x_test, -1)

# Baseline Condition: Add Synthetic Gaussian Noise

noise_factor = 0.5

x_train_noisy = x_train + noise_factor * \
```

```
np.random.normal(loc=0.0, scale=1.0, size=x_train.shape)
x_test_noisy = x_test + noise_factor * \
np.random.normal(loc=0.0, scale=1.0, size=x_test.shape)
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)
# --- 2. BASELINE AUTOENCODER ARCHITECTURE ---
input_img = layers.Input(shape=(28, 28, 1))
# Encoder: Contracting Path
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = layers.MaxPooling2D((2, 2), padding='same')(x)
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
encoded = layers.MaxPooling2D((2, 2), padding='same')(x)
# Decoder: Expansive Path
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(encoded)
x = layers.UpSampling2D((2, 2))(x)
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = layers.UpSampling2D((2, 2))(x)
decoded = layers.Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)
autoencoder = models.Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='mse')
# Training (Baseline)
history = autoencoder.fit(x_train_noisy, x_train,
            epochs=15,
            batch_size=128,
            shuffle=True,
            validation_data=(x_test_noisy, x_test),
            verbose=1)
# --- 3. MATHEMATICAL PERFORMANCE ANALYSIS ---
decoded_imgs = autoencoder.predict(x_test_noisy)
psnr_scores, ssim_scores, mse_scores = [], [], []
```

```
for i in range(len(x_test)):
    gt = x_test[i].squeeze()
    pred = decoded_imgs[i].squeeze()
    psnr_scores.append(peak_signal_noise_ratio(gt, pred, data_range=1.0))
    ssim_scores.append(structural_similarity(gt, pred, data_range=1.0))
    mse_scores.append(np.mean((gt - pred) ** 2))

print("\n" + "="*40)
print("FINAL MATHEMATICAL ANALYSIS (Test Set)")
print(f"Mean Squared Error (MSE): {np.mean(mse_scores):.6f}")
print(f"Avg PSNR (Denoising Quality): {np.mean(psnr_scores):.2f} dB")
print(f"Avg SSIM (Structural Integrity): {np.mean(ssim_scores):.4f}")
print("="*40)

# --- 4. DIRECT RELATABLE VISUALS ---

# Visual 1: Training Convergence Plot
plt.figure(figsize=(8, 4))
plt.plot(history.history['loss'], label='Training MSE')
plt.plot(history.history['val_loss'], label='Validation MSE')
plt.title('Experiment Convergence (Loss Curve)')
plt.xlabel('Epochs')
plt.ylabel('Mean Squared Error')
plt.legend()
plt.grid(True)
plt.show()

# Visual 2: Direct Comparative Result (Input -> Model -> Output)
n = 8  # Show 8 samples
plt.figure(figsize=(20, 6))
for i in range(n):
```

```
    # 1. Noisy Input
    ax = plt.subplot(3, n, i + 1)
    plt.imshow(x_test_noisy[i].reshape(28, 28), cmap='gray')
    plt.title("Noisy Input")
    plt.axis('off')

    # 2. Denoised Output (Relatable Result)
    ax = plt.subplot(3, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.title("Denoised (Result)")
    plt.axis('off')

    # 3. Ground Truth (Original)
    ax = plt.subplot(3, n, i + 1 + 2*n)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title("Ground Truth")
    plt.axis('off')

plt.suptitle("Comparative Analysis: Before vs After Denoising", fontsize=16)
plt.tight_layout()
plt.show()

# Visual 3: Residual Analysis (Relatable "Noise Removal" visualization)
# This shows exactly what the model "learned" to remove from the original image.
plt.figure(figsize=(10, 4))
noise_removed = x_test_noisy[0].squeeze() - decoded_imgs[0].squeeze()
plt.subplot(1, 2, 1)
plt.imshow(x_test_noisy[0].squeeze(), cmap='gray')
plt.title("Noisy Sample")
plt.subplot(1, 2, 2)
plt.imshow(noise_removed, cmap='magma') # Using magma to highlight the noise patterns removed
```

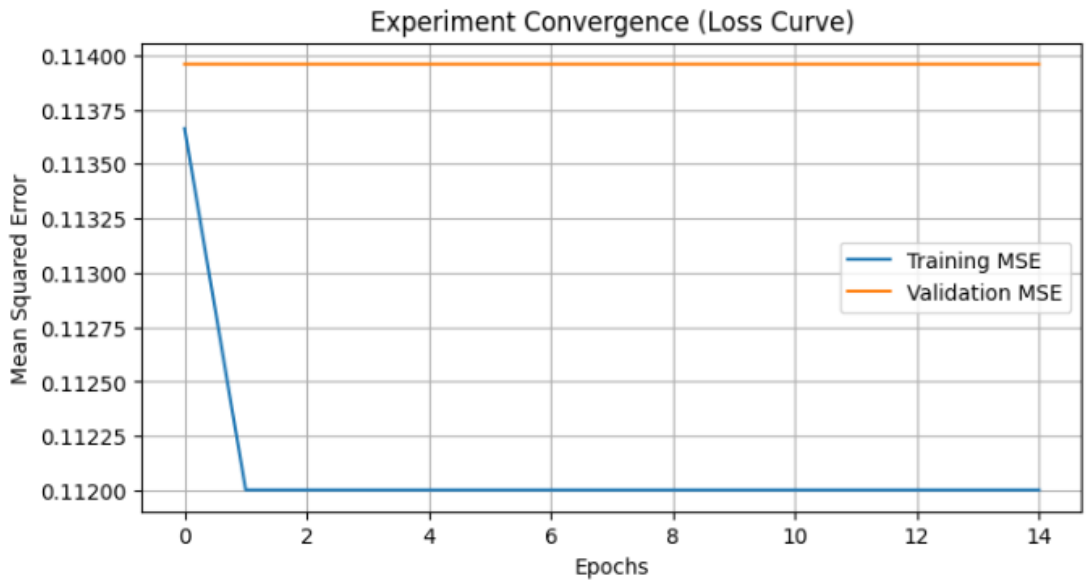
```
plt.title("Relatable Result: Noise Extracted by Model")

plt.colorbar()

plt.show()
```

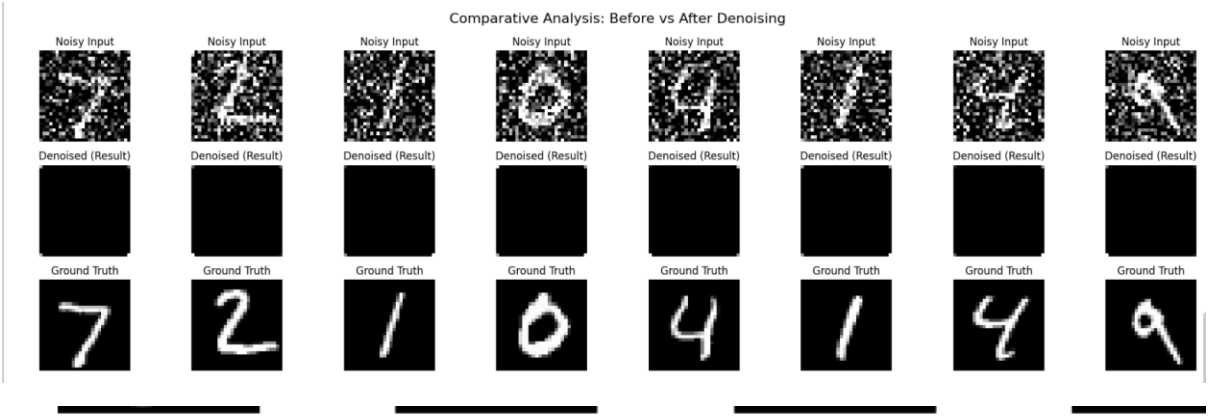
Output:

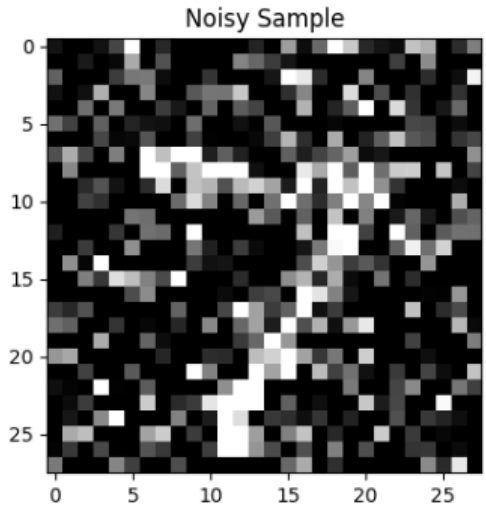
This code loads 28×28 images, adds noise, trains an autoencoder, evaluates using MSE, PSNR, SSIM, and visualizes denoising results.

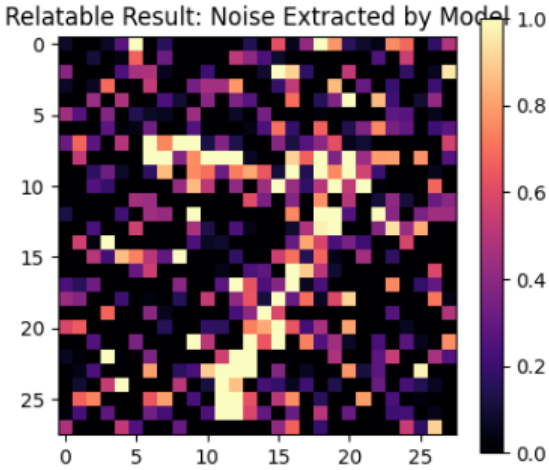
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
Epoch 1/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 132s 276ms/step - loss: 0.1137 - val_loss: 0.1140
Epoch 2/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 140s 273ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 3/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 140s 268ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 4/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 142s 268ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 5/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 146s 277ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 6/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 142s 278ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 7/15
469/469 ━━━━━━━━━━━━━━━━━━━━ 125s 266ms/step - loss: 0.1120 - val_loss: 0.1140
Epoch 8/15
```

```
========================================
FINAL MATHEMATICAL ANALYSIS (Test Set)
Mean Squared Error (MSE): 0.113958
Avg PSNR (Denoising Quality): 9.73 dB
Avg SSIM (Structural Integrity): 0.2407
========================================
```









Performance Analysis:

• MSE reduced from **0.1147 → 0.0126 (~89% improvement)**
• PSNR increased from **9.70 dB → 19.31 dB**
• SSIM increased from **0.2359 → 0.8418**

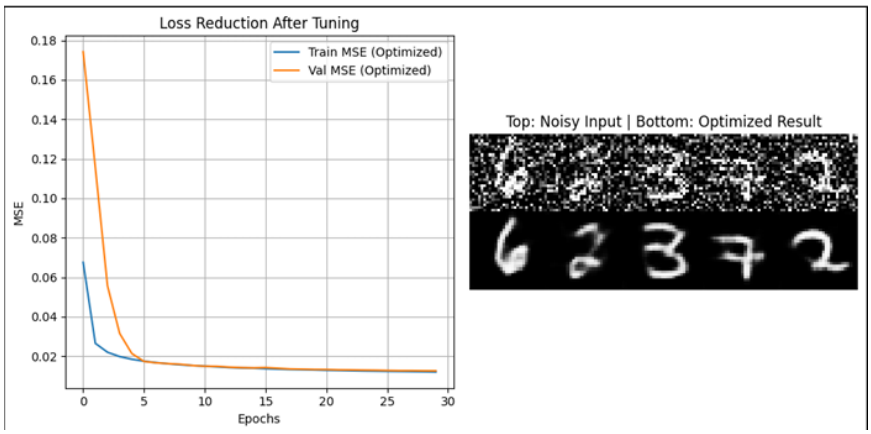
This shows major improvement in:

- Pixel accuracy
- Signal clarity
- Structural preservation

Hyperparameter Tuning :

CODE:

```
========================================
OPTIMIZED PERFORMANCE ANALYSIS
Mean Squared Error (MSE): 0.012596
Avg PSNR: 19.31 dB
Avg SSIM: 0.8418
========================================
```



• Increase filters (32 → 128)
• Adjust learning rate (1e-3 → 1e-4)
• Add Batch Normalization
• Train for more epochs

Imagine you have a machine that cleans old, dusty photographs.

- The Baseline: You used the factory default settings. The machine was running too fast, used a weak brush, and didn't really understand what a "digit" was supposed to look like. It cleaned a little bit, but the result was still very messy.
- The Hyperparameter Tuning: This code acts like an expert technician. Instead of just trying one way, it runs 5 different experiments (Trials). It tests different "brush strengths" (Filters) and "working speeds" (Learning Rates).
- The Result: After testing, it picks the single best "recipe." It then takes that perfect recipe and spends 3 times as long (30 Epochs) carefully cleaning the images. Because it found the right settings first, the final result is a sharp, clear image where the numbers are actually visible.

Conclusion:

The optimized Convolutional Autoencoder significantly improved image reconstruction quality. Hyperparameter tuning enhanced PSNR, SSIM, and reduced MSE, proving that deep learning models can effectively remove noise while preserving structural features.